

UNIT–II: Process Management

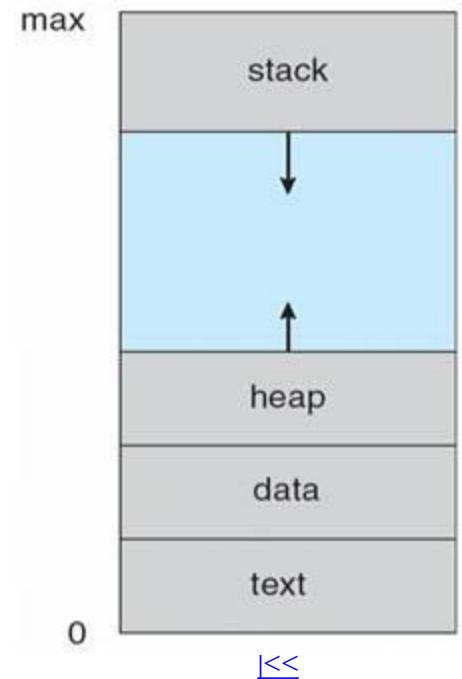
- 1. Process [Concepts](#)**
- 2. Process Scheduling**
- 3. Operation on Processes**
- 4. CPU [Scheduling](#)**
- 5. Scheduling [Criteria](#)**
- 6. Scheduling [Algorithms](#)**
 - **First Come First Serve (FCFS)**
 - **Shortest Job First (SJF)**
 - **Priority Scheduling**
 - **Round Robin (RR)**
- 7. Multilevel Queue Scheduling**
- 8. Case Studies**
 - **Unix**
 - **Linux**
 - **Windows**
- **Exam [Questions](#)**

Process Concepts

- Process
- Process States
- Process Control Block (PCB)
- Switch from Process to Process
- Process Scheduling Queues
- Schedulers

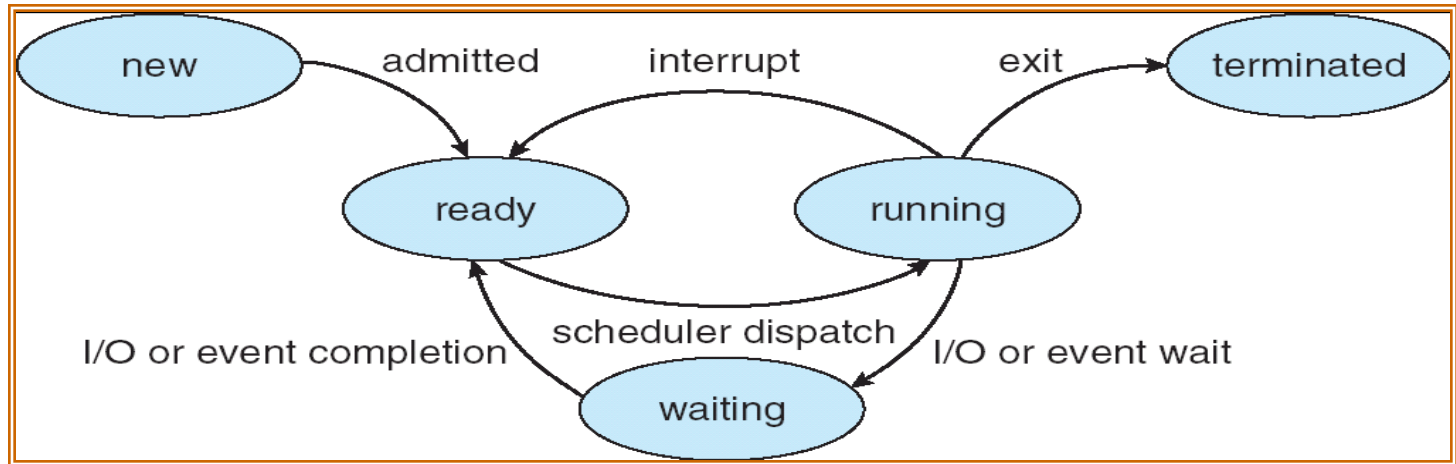
Process

- **A program in execution.**
- **Process execution must progress in Sequential fashion.**
- **A process is more than a program code, which is know as text section.**
- **includes:**
 - **Program Counter (current activity and contents of reg.)**
 - **Stack(temporary data)**
 - **Data Section(global variables)**
 - **Heap.**



Process States

As a process executes, it changes *state*

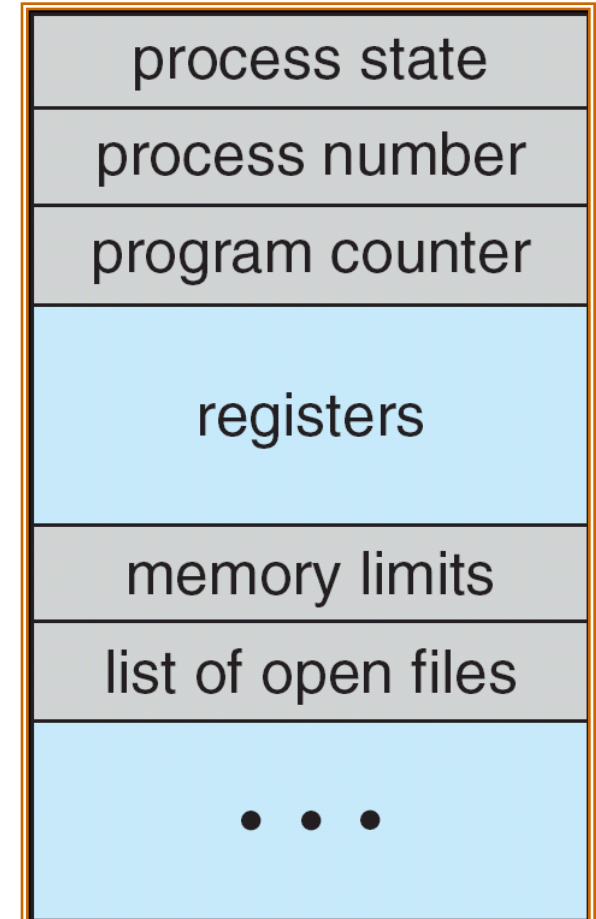


- **new**: The process is being created
- **ready**: The process is waiting to be assigned to a processor
- **running**: Instructions are being executed
- **waiting**: The process is waiting for some event to occur
- **terminated**: The process has finished execution

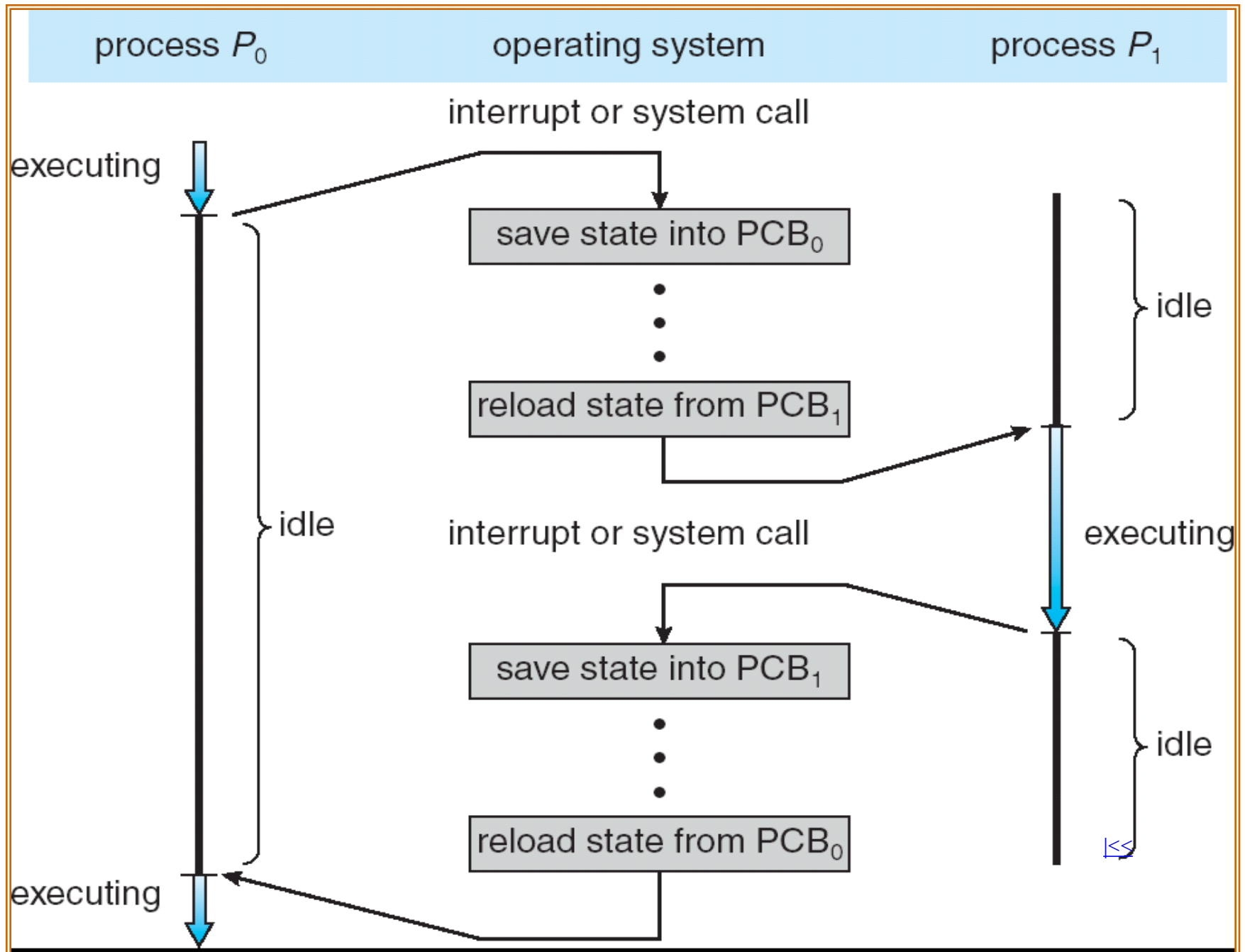
Process Control Block (PCB)

Information associated with each process

- Process State
- Process number
- Program Counter
- CPU Registers
- Memory Management information
- I/O Status information
- CPU Scheduling information
- Accounting information



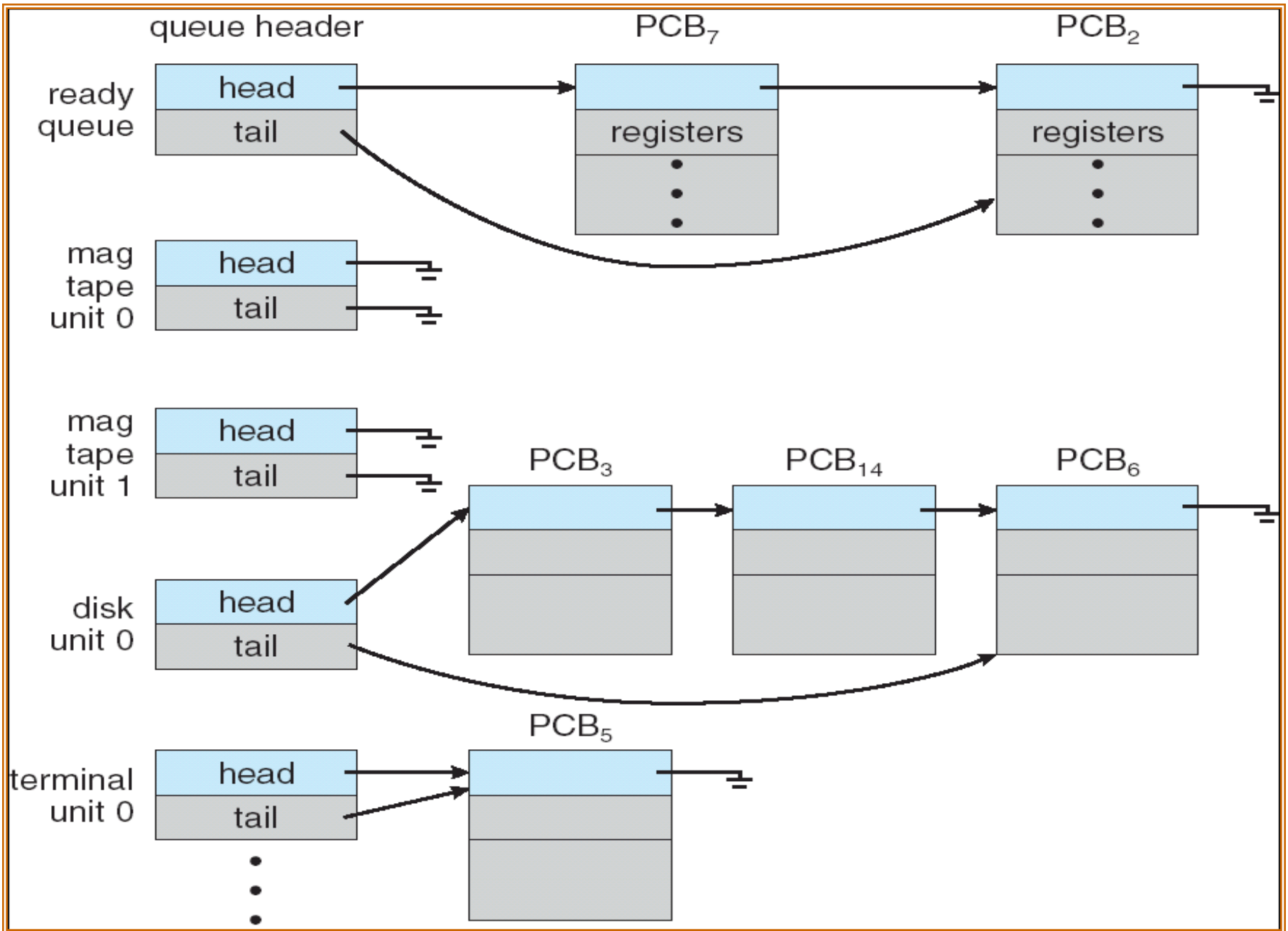
CPU Switch From Process to Process



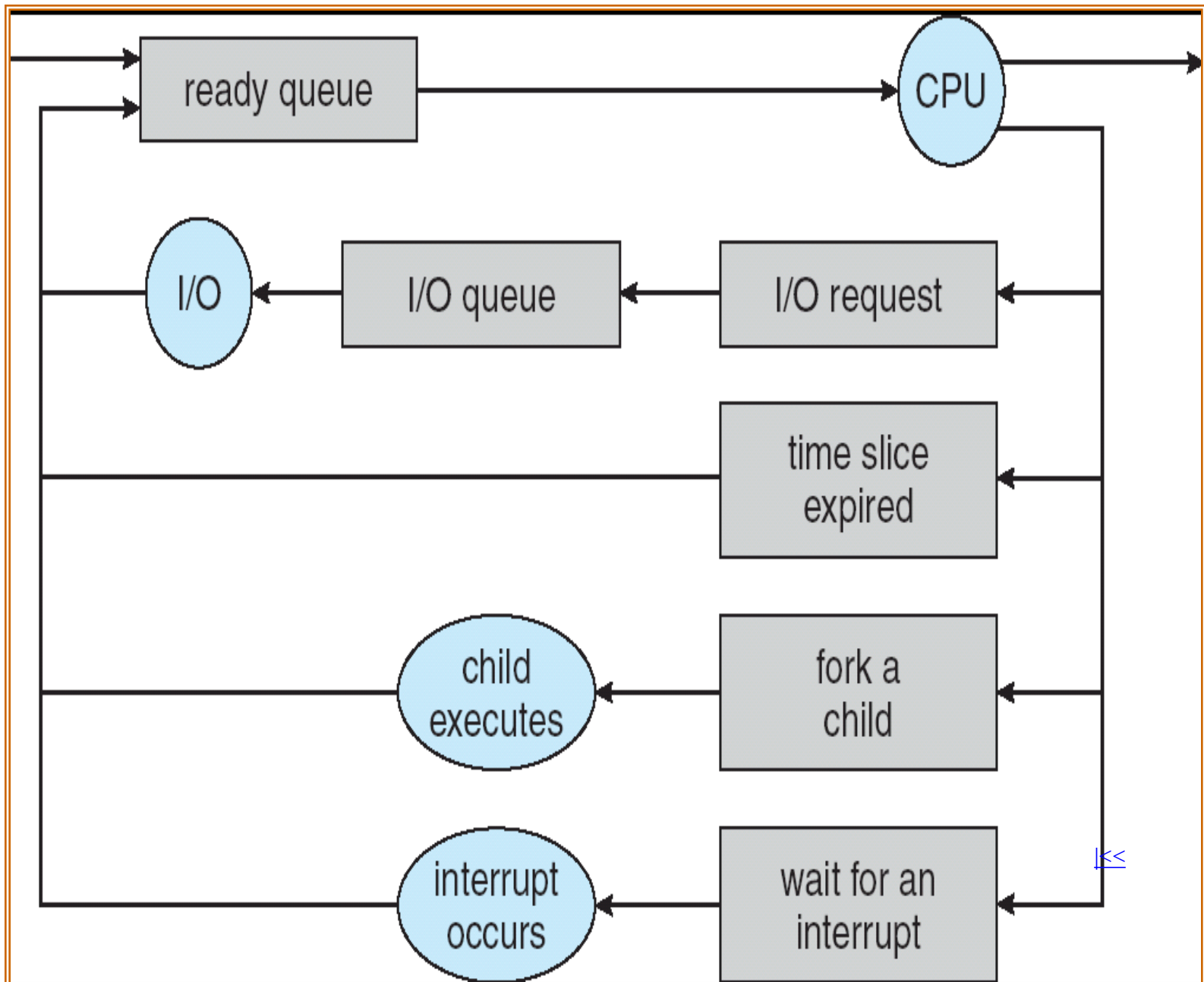
Process Scheduling Queues

- **Job queue**
 - Set of all processes in the system
- **Ready queue**
 - Set of all processes residing in main memory, ready and waiting to execute
- **Device queues**
 - Set of processes waiting for an I/O device
- Processes migrate among the various queues

Ready Queue and Various I/O Device Queues



Representation of Process Scheduling



Schedulers

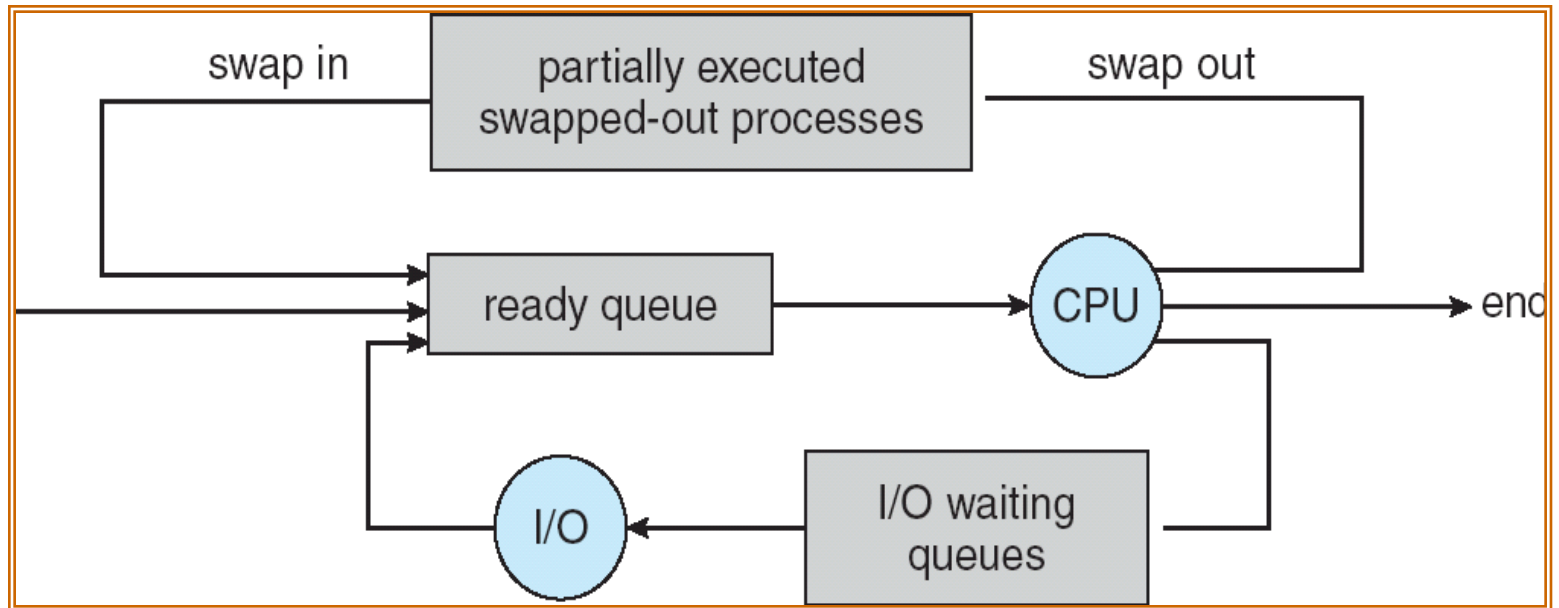
- Question: Draw the process states and...
- **Long-term Scheduler** (or Job scheduler)
 - Selects which processes should be brought into the ready queue
 - Decision is based on CPU scheduling algorithms like FCFS, priority, execution time or Input/Output requirements.
 - This executes relatively infrequently.
- **Short-term Scheduler** (or CPU scheduler or Dispatcher)
 - Selects from among the processes that are ready to execute and allocates the CPU to one of them.
 - invoked whenever an event occurs, leading to interruption. Ex: clock interrupts, I/O interrupts, operating system calls, signals, etc.
 - It must select a new process for the CPU frequently. It must be very fast.
- Question: Locate the position where Long Term and Short Term Scheduler execute in the above diagram?

Addition of Medium Term Scheduling

- The medium-term scheduler temporarily removes processes from main memory and places them in secondary memory , like hard disk drive, or vice-versa, referred to as "swapping out" or "swapping in".
- It may decide to swap out a process when it
 - inactive for some time,
 - has a low priority,
 - page faulting frequently,
 - is taking up a large amount of memory
- In order to free up main memory for other processes, swapping the process back in later when more memory is available, or when the process has been unblocked and is no longer waiting for a resource.

Question: Locate where Medium term scheduler works?

Contd..



Operation on Processes

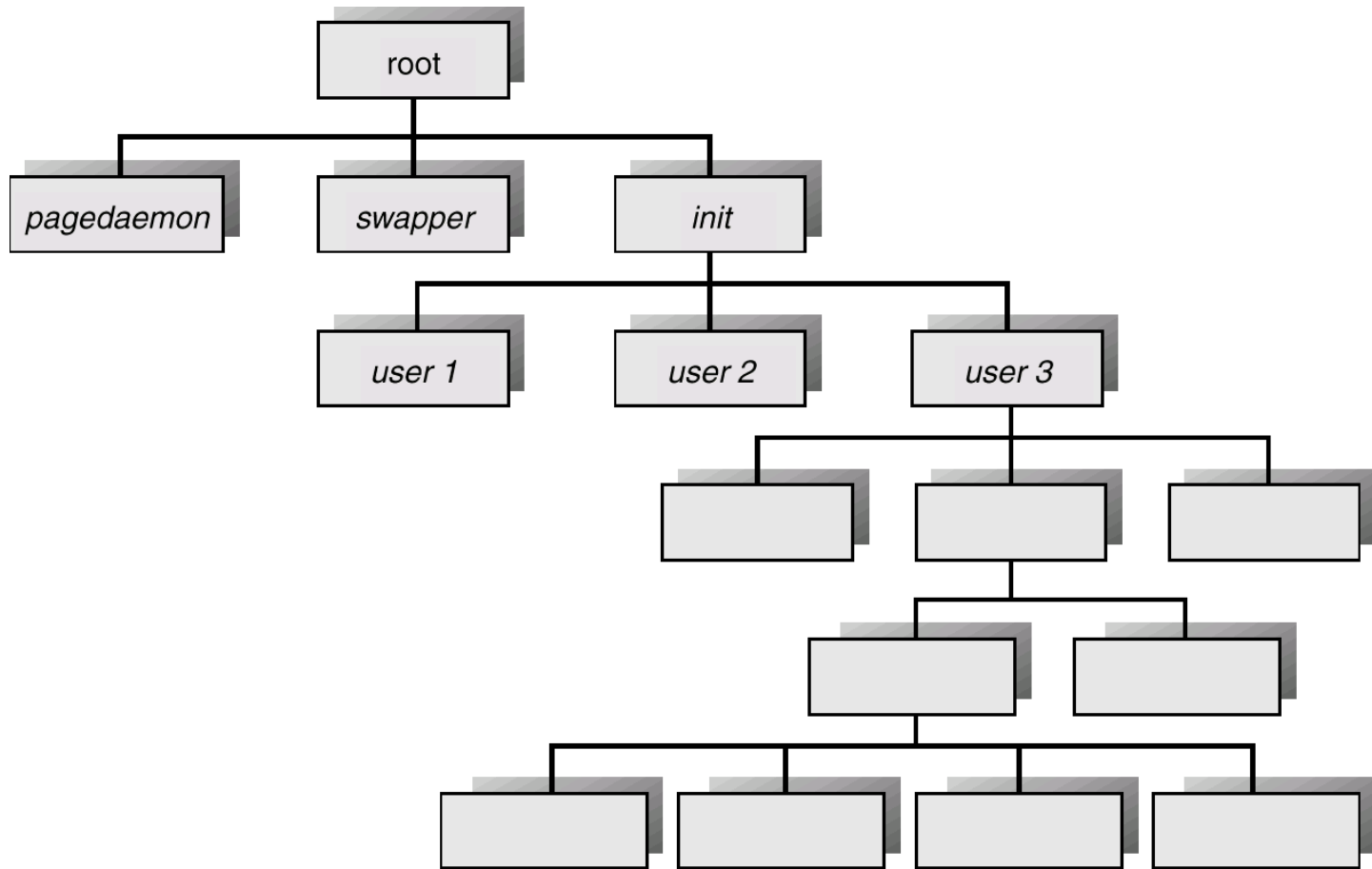
- **Process Creation**

- Parent process creates children processes, which, in turn create other processes, forming a tree of processes.
- Resource sharing
 - Parent and children share all resources.
 - Children share subset of parent's resources.
 - Parent and child share no resources.
- Execution
 - Parent and children execute concurrently.
 - Parent waits until children terminate.

Process Creation (Cont.)

- Address space
 - Child is duplicate of parent(same program and data as parent).
 - Child has a new program loaded into it.
- UNIX examples
 - **fork** system call creates new process
 - **exec()** system call used after a **fork** to replace the process's memory space with a new program.

A Tree of Processes On A Typical UNIX System



Process Termination

- Process executes last statement and asks the operating system to decide it (**exit**).
 - Output data from child to parent (via **wait**).
 - Process resources are deallocated by operating system.
- Parent may terminate execution of children processes (**abort**).
 - Child has exceeded allocated resources.
 - Task assigned to child is no longer required.
 - Parent is exiting.
 - Operating system does not allow child to continue if its parent terminates.
- Cascading termination.

CPU Scheduling

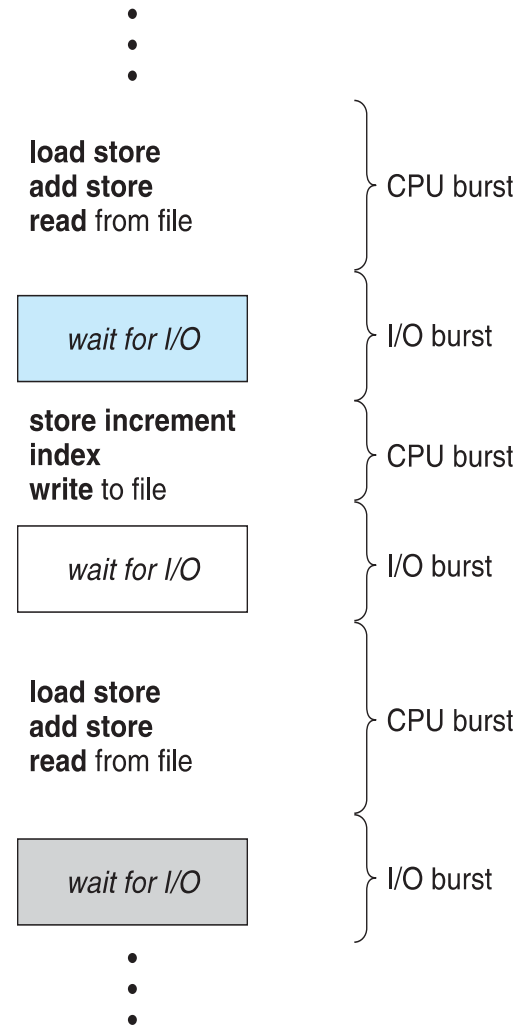
- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms

Objectives

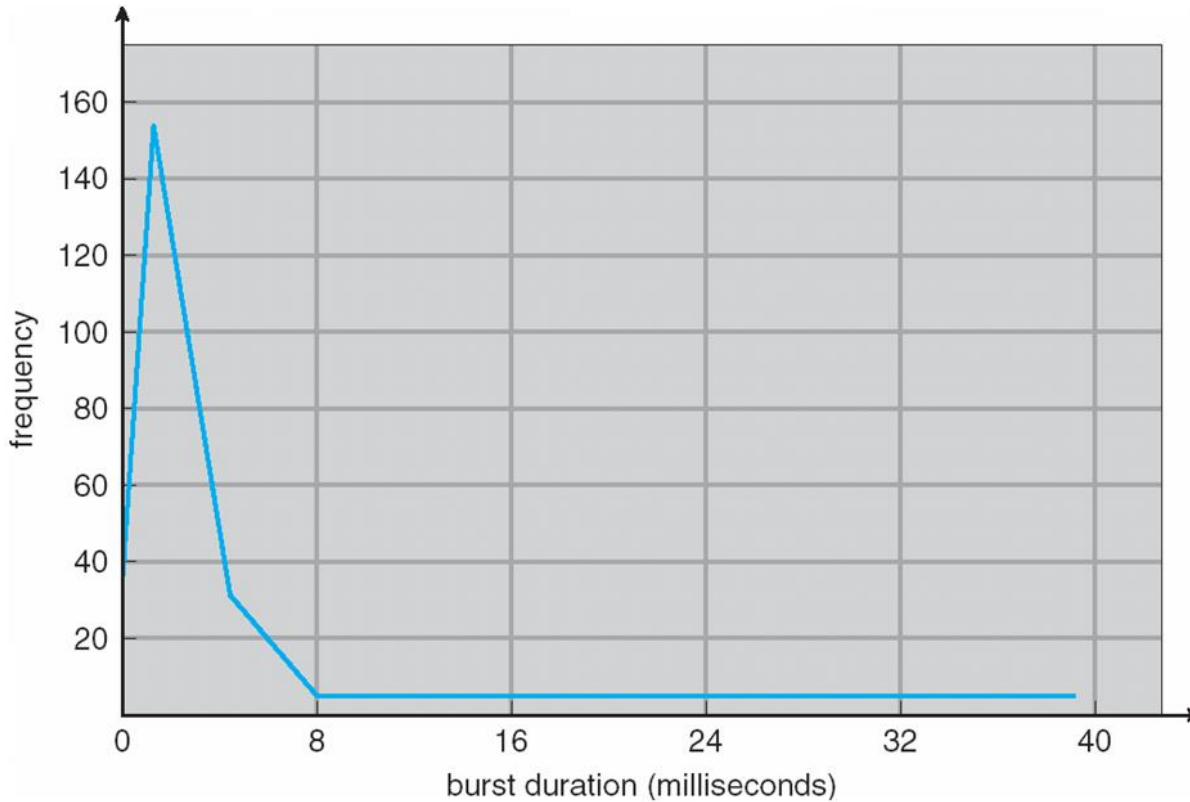
- To introduce CPU scheduling, which is the basis for multiprogrammed operating systems
- To describe various CPU-scheduling algorithms
- To discuss evaluation criteria for selecting a CPU-scheduling algorithm for a particular system
- To examine the scheduling algorithms of several operating systems

Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- CPU burst distribution is of main concern



Histogram of CPU-burst Times



CPU Scheduler

- **Short-term scheduler** selects from among the processes in ready queue, and allocates the CPU to one of them
 - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state
 2. Switches from running to ready state
 3. Switches from waiting to ready
 4. Terminates
- Scheduling under 1 and 4 is **nonpreemptive** or **cooperative**
- All other scheduling is **preemptive**
 - Consider access to shared data(incurs more cost)
 - Consider preemption while in kernel mode(affects design of OS)
 - Consider interrupts occurring during crucial OS activities

Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – no. of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process. (Time of submission of a process to the time of completion)
- **Waiting time** – amount of time a process spends waiting in the ready queue. Waiting time is the sum of the periods spent in the ready queue.
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced. It is the time it takes to start responding, not the time it takes to output the response. (for time-sharing environment)

Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Scheduling Algorithms

- FCFS Scheduling
- SJF Scheduling
 - Non-Preemptive
 - Preemptive
- Priority Scheduling
- Round Robin
- Multilevel Queue Scheduling

First- Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case
- **Convoy effect** - short process behind long process
 - Consider one CPU-bound and many I/O-bound processes

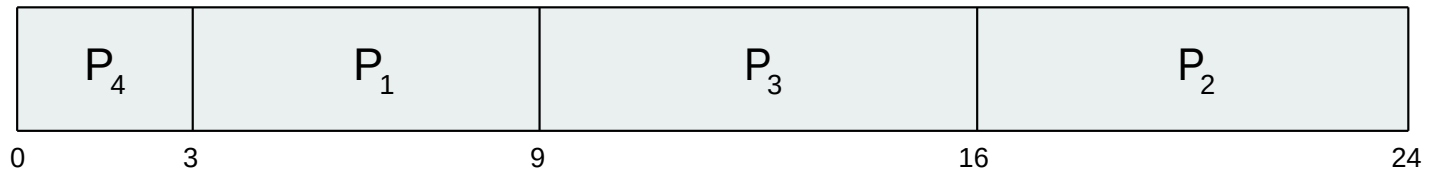
Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst
 - Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
 - The difficulty is knowing the length of the next CPU request
 - Could ask the user

Example of SJF

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

- SJF scheduling chart

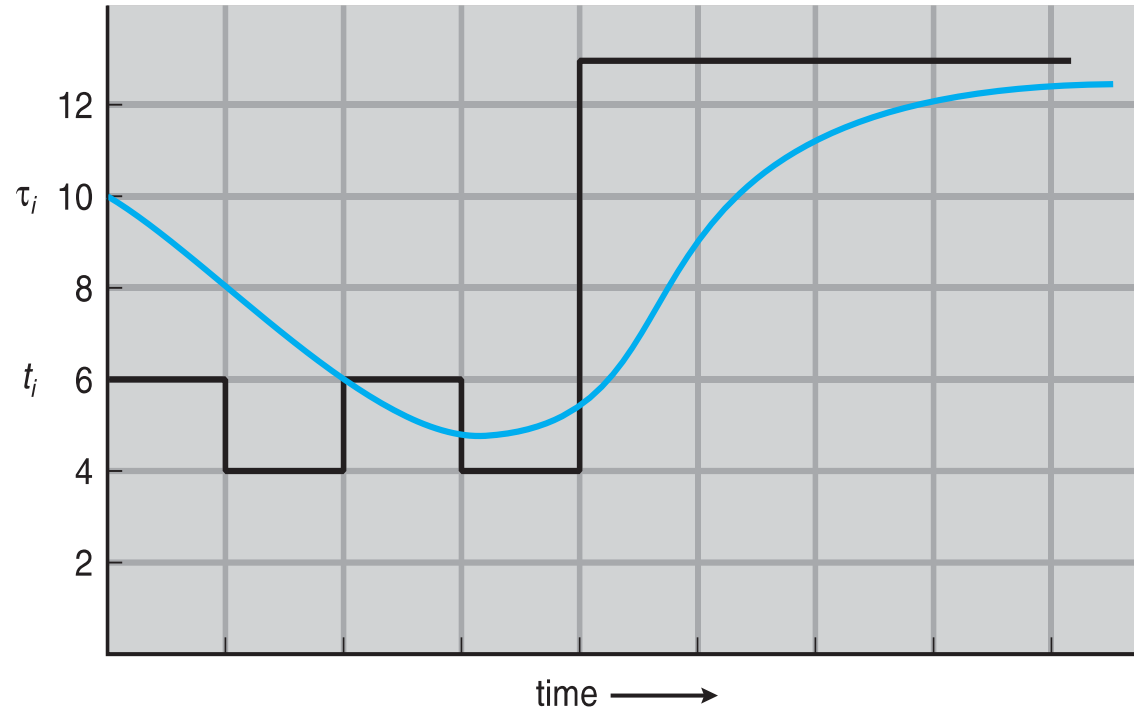


- Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$

Determining Length of Next CPU Burst

- Can only estimate the length – should be similar to the previous one
 - Then pick process with shortest predicted next CPU burst
- Can be done by using the length of previous CPU bursts, using exponential averaging
 1. t_n = actual length of n^{th} CPU burst
 2. τ_{n+1} = predicted value for the next CPU burst
 3. $\alpha, 0 \leq \alpha \leq 1$
 4. Define: $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.
- Commonly, α set to $\frac{1}{2}$
- Preemptive version called **shortest-remaining-time-first**

Prediction of the Length of the Next CPU Burst



CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	9	11	12	...

Examples of Exponential Averaging

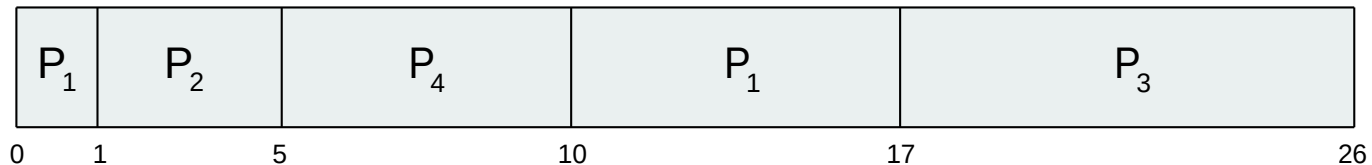
- $\alpha = 0$
 - $\tau_{n+1} = \tau_n$
 - Recent history does not count
- $\alpha = 1$
 - $\tau_{n+1} = \alpha t_n$
 - Only the actual last CPU burst counts
- If we expand the formula, we get:
$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \dots$$
$$+ (1 - \alpha)^j \alpha t_{n-j} + \dots$$
$$+ (1 - \alpha)^{n+1} \tau_0$$
- Since both α and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor

Example of Shortest-remaining-time-first

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- Preemptive* SJF Gantt Chart



- Average waiting time = $[(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5$ msec

Priority Scheduling

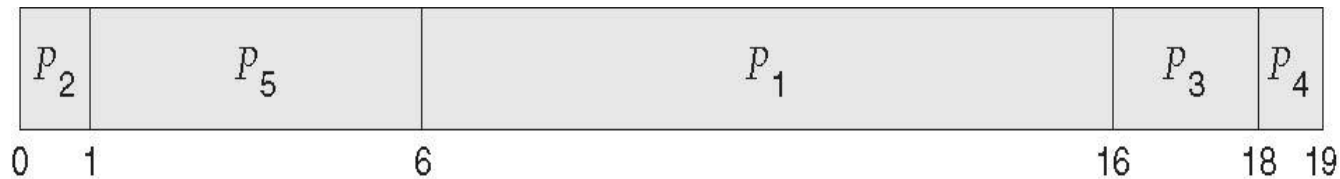
- A priority No. (Integer) is associated with each process
- The CPU is allocated to the process with the highest priority
 - Smallest integer $\equiv$ highest priority
 - Preemptive
 - Non-preemptive
- SJF is a priority scheduling where priority is the predicted next CPU burst time
- Problem $\equiv$ Starvation
 - Low priority processes may never execute
- Solution $\equiv$ Aging
 - As time progresses increase the priority of the process

Example	Proc	Arrival	Burst	Priority
	P ₁	0	10	3
	P ₂	1	1	1
	P ₃	2	2	3
	P ₄	4	1	4
	P ₅	8	5	2

Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

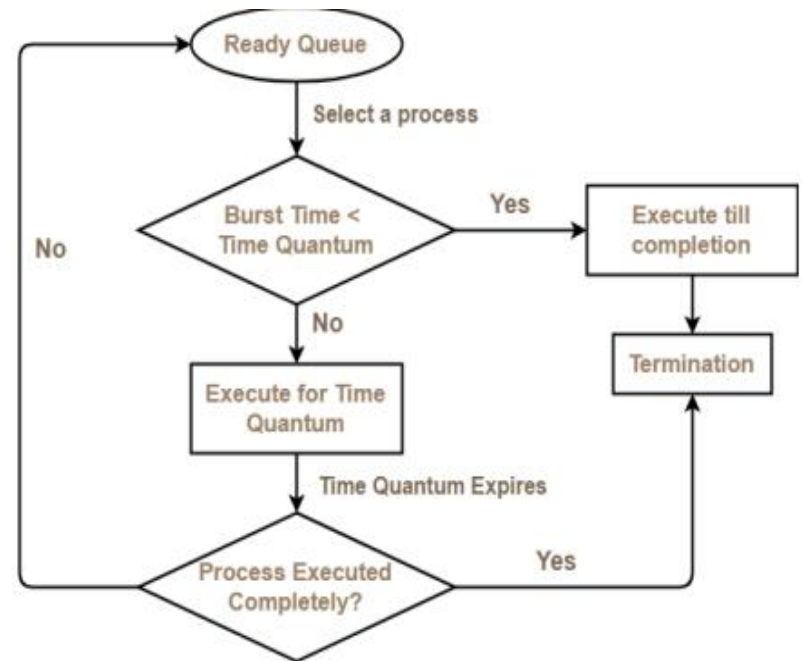
- Priority scheduling Gantt Chart



- Average waiting time = 8.2 msec

Round Robin (RR)

- Each process gets a small unit of CPU time (*time quantum*), usually 10-100 milliseconds.
- After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.

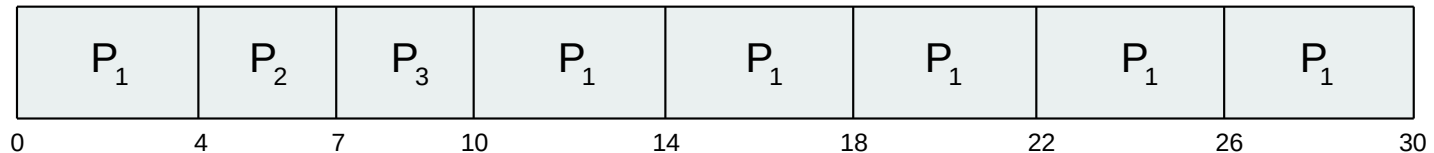


Round Robin Scheduling

Example of RR with Time Quantum = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- The Gantt chart is:

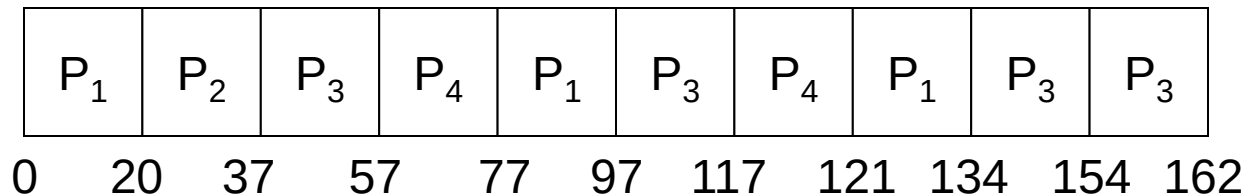


- Typically, higher average turnaround than SJF, but better **response**
- q should be large compared to context switch time
- q usually 10ms to 100ms, context switch < 10 usec

Example of RR with Time Quantum = 20

<u>Process</u>	<u>Burst Time</u>
P_1	53
P_2	17
P_3	68
P_4	24

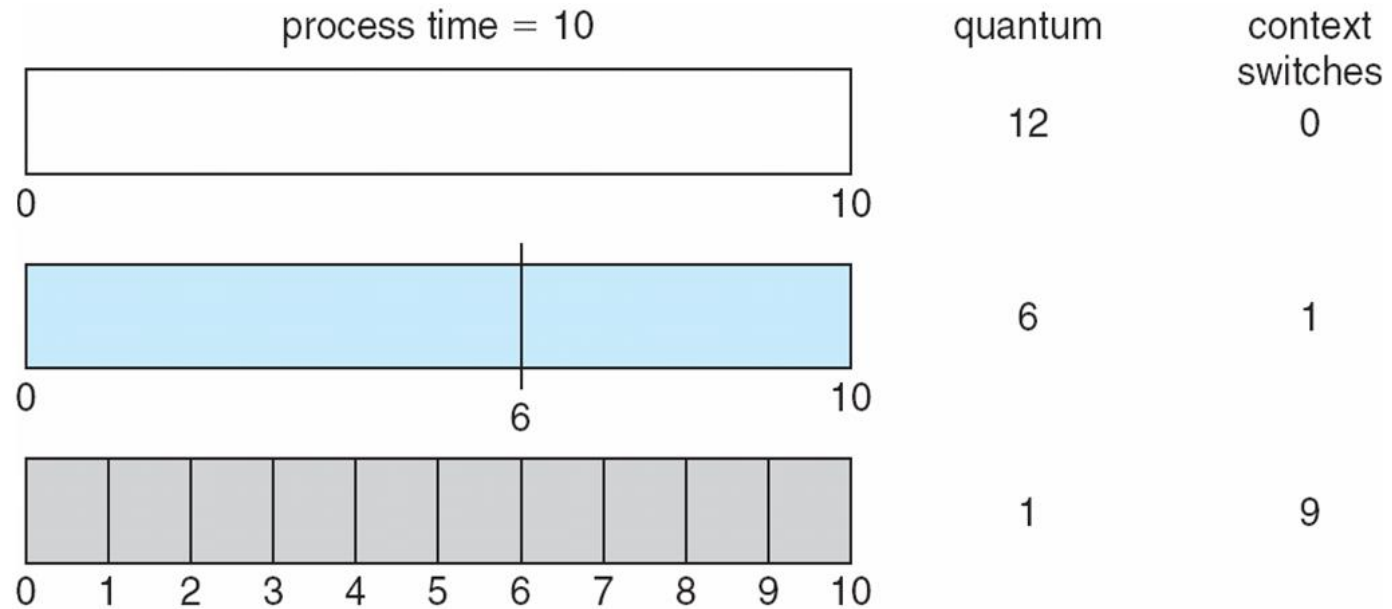
Gantt chart:



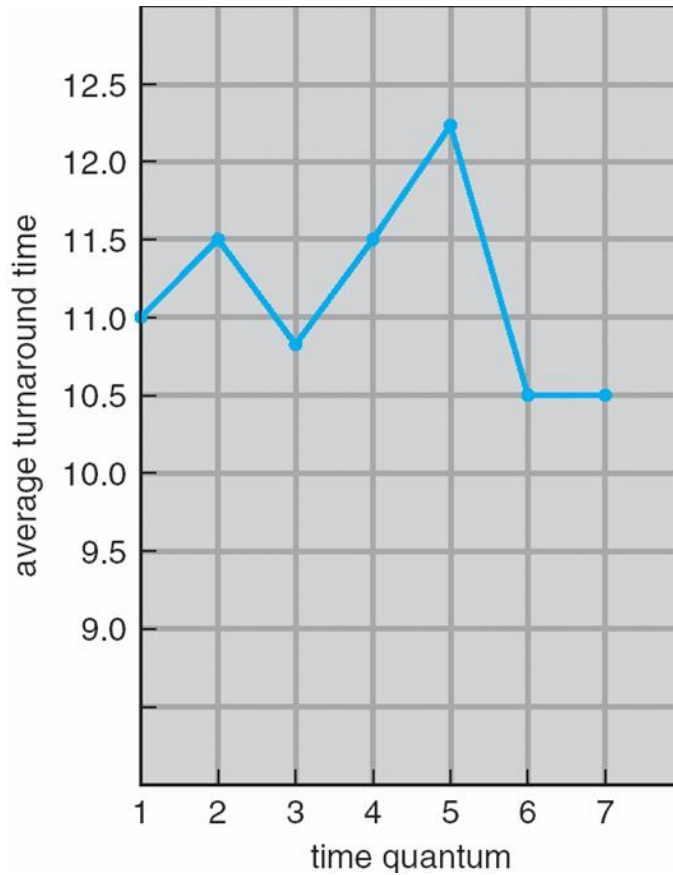
Average waiting time

$$\begin{aligned} &= [(77 + 24) + (20) + (37 + 40 + 17) + (57 + 40)] / 4 \\ &= [101 + 20 + 94 + 97] / 4 \\ &= 312 / 4 \\ &= 78 \end{aligned}$$

Time Quantum and Context Switch Time



Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

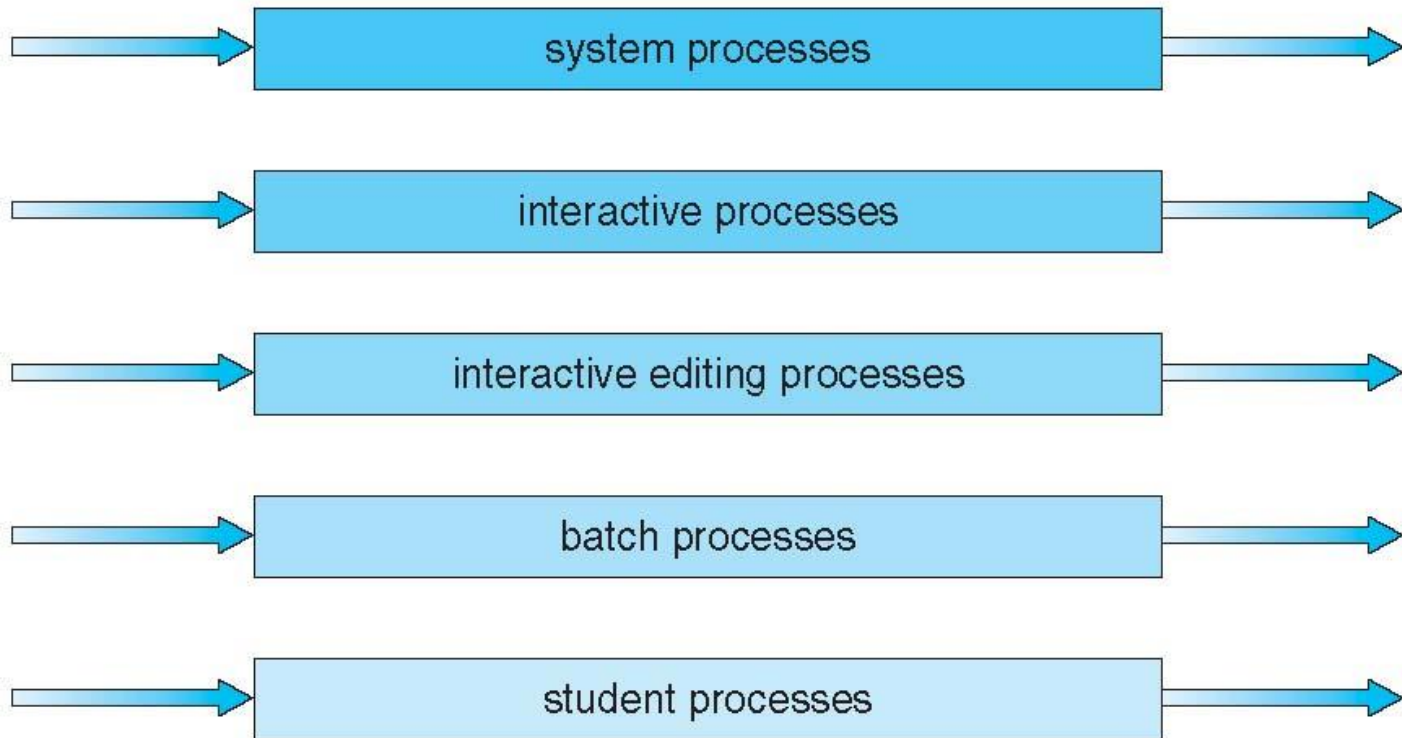
80% of CPU bursts should be shorter than q

Multilevel Queue

- Ready queue is partitioned into separate queues, eg:
 - **foreground** (interactive)
 - **background** (batch)
- Processes are permanently assigned to one queue
- Each queue has its own scheduling algorithm:
 - foreground – RR
 - background – FCFS
- Scheduling must be done between the queues:
 - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
 - 20% to background in FCFS

Multilevel Queue Scheduling

highest priority



lowest priority

Multilevel Feedback Queue

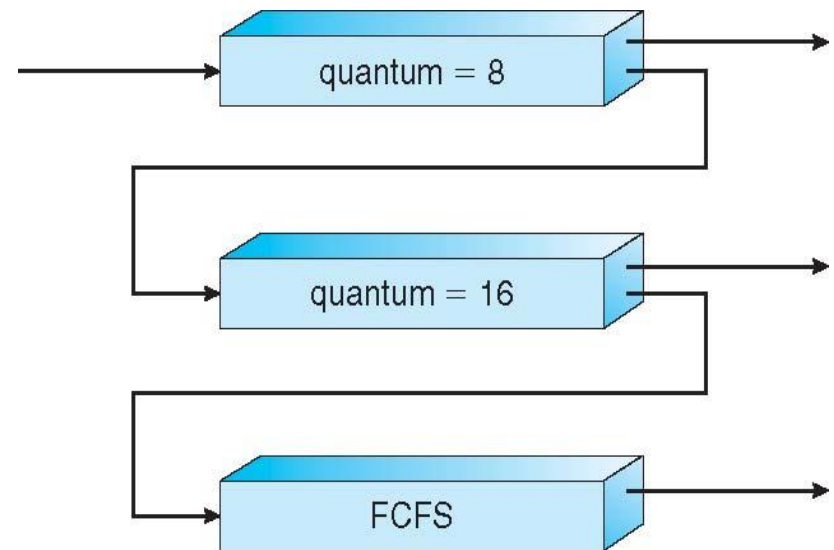
- In a multi-level queue-scheduling algorithm, processes are permanently assigned to a queue.
- Idea: Allow processes to move among various queues.
- Examples
 - If a process in a queue dedicated to interactive processes consumes too much CPU time, it will be moved to a (lower-priority) queue.
 - A process that waits too long in a lower-priority queue may be moved to a higher-priority queue.

Multilevel Feedback Queue

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service

Example of Multilevel Feedback Queue

- Three queues:
 - Q_0 – RR with time quantum 8 milliseconds
 - Q_1 – RR time quantum 16 milliseconds
 - Q_2 – FCFS



- Scheduling
 1. The scheduler first executes all processes in Queue 0.
 2. Only when queue 0 is empty will it execute processes in queue 1.
 3. Similarly, processes in queue 2 will only be executed if queue 0 and 1 are empty.
 4. A process that arrive for Q1 will preempt a process in Q2.
 5. A process in Q1 will in turn be preempted by a process arriving for Q0

- A process entering the Ready queue is put in Q0 is given a TQ of 8 ms.
- If it does not finish within this time, it is moved to tail of Queue 1. If Q0 is empty, the process at the head of queue 1 is given a TQ of 16 ms.
- If it does not complete, it is preempted and is put into Q2. Processes in queue 2 are run on an FCFS basis but are run only when Q0 and Q1 are empty.

Exercise: Five batch jobs P1 through P5 arrive for execution at times indicated. Each process has a total CPU time requirement as listed below:

Using FCFS, determine the *average turnaround* and *average waiting times*. Also draw the *Gantt charts*.

Process	Arrival time	CPU burst
P1	2	10
P2	1	2
P3	0	5
P4	4	6
P5	3	4

Exercise: Five batch jobs P1 through P5 arrive for execution at times indicated. Each process has a total CPU time requirement as listed below:

Using Preemptive SJF, determine the *average turnaround* and *average waiting times*. Also draw the *Gantt charts*.

Process	Arrival time	CPU burst
P1	2	10
P2	1	2
P3	0	5
P4	4	6
P5	3	4

More SJF Examples

1. SJF non-preemptive	Proc	Arrives	Burst
	P ₁	0	8
	P ₂	1	4
	P ₃	2	9
	P ₄	3	5

And then preemptive

2. SJF non-preemptive	Proc	Arrives	Burst
	P ₁	1	2
	P ₂	0	7
	P ₃	2	7
	P ₄	5	3
	P ₅	6	1

And then preemptive

More Priority Examples

- Example

	Proc	Arrival	Burst	Priority
	P_1	0	6	5
	P_2	2	2	3
	P_3	3	3	4
	P_4	9	3	2
	P_5	10	1	1

Five batch jobs P1 through P5 arrive for execution at times indicated. Each process has a total CPU time requirement as listed below:

Using RR with $Q = 3$ units, determine the *average turnaround* and *average waiting times*. Also draw the *Gantt charts*.

Process	Arrival time	CPU burst
P1	2	10
P2	1	2
P3	0	5
P4	4	6
P5	3	4

Multilevel Queue Examples

- ML queue, 2 levels
 - RR @ 10 units
 - FCFS
 - RR gets priority over FCFS
- | Proc | Arrival | Burst | Queue |
|-------|---------|-------|-------|
| P_1 | 0 | 12 | FCFS |
| P_2 | 4 | 12 | RR |
| P_3 | 8 | 8 | FCFS |
| P_4 | 20 | 10 | RR |
- Non-preemptive and preemptive

Example of Multilevel Feedback Queue

- Three queues:
 - Q_0 – time quantum 8 milliseconds
 - Q_1 – time quantum 16 milliseconds
 - Q_2 – FCFS
- Scheduling
 - A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1 .
 - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2 .

Multilevel Feedback Queue Example

- Three levels
 - RR at 8 units
 - RR at 16 units
 - FCFS, active
- *Proc* *Arrival* *Burst*

P_1	0	32
P_2	10	12
P_3	30	10
- Non-preemptive and preemptive

Operating System Examples

- Windows XP scheduling
- Linux scheduling

Windows Scheduling

- Windows uses priority-based preemptive scheduling
- Highest-priority thread runs next
- *Dispatcher* is scheduler
- Thread runs until
 - (1) blocks,
 - (2) uses time slice,
 - (3) preempted by higher-priority thread
- Real-time threads can preempt non-real-time
- 32-level priority scheme
- **Variable class** is 1-15, **real-time class** is 16-31
- Priority 0 is memory-management thread
- Queue for each priority
- If no run-able thread, runs **idle thread**

Windows Priority Classes

- Win32 API identifies several priority classes to which a process can belong
 - `REALTIME_PRIORITY_CLASS`, `HIGH_PRIORITY_CLASS`, `ABOVE_NORMAL_PRIORITY_CLASS`, `NORMAL_PRIORITY_CLASS`, `BELOW_NORMAL_PRIORITY_CLASS`, `IDLE_PRIORITY_CLASS`
 - All are variable except `REALTIME`
- A thread within a given priority class has a relative priority
 - `TIME_CRITICAL`, `HIGHEST`, `ABOVE_NORMAL`, `NORMAL`, `BELOW_NORMAL`, `LOWEST`, `IDLE`
- Priority class and relative priority combine to give numeric priority
- Base priority is `NORMAL` within the class
- If quantum expires, priority lowered, but never below base
- If wait occurs, priority boosted depending on what was waited for
- Foreground window given 3x priority boost

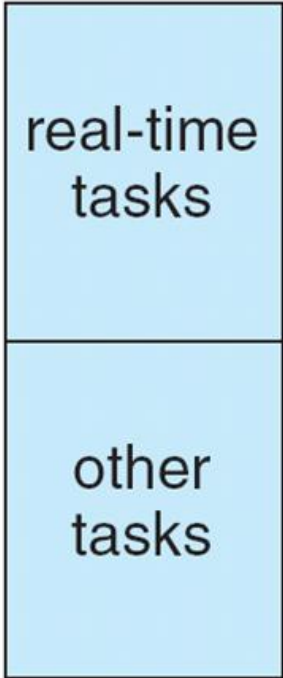
Windows XP Priorities

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

Linux Scheduling

- Constant order $O(1)$ scheduling time
- Preemptive, priority based
- Two priority ranges: time-sharing and real-time
- **Real-time** range from 0 to 99 and **nice** value from 100 to 140
- Map into global priority with numerically lower values indicating higher priority
- Higher priority gets larger q
- Task run-able as long as time left in time slice (**active**)
- If no time left (**expired**), not run-able until all other tasks use their slices
- The kernel maintains a list of all runnable tasks in a **runqueue**
- Because of its support for SMP, each processor maintains its own queue and schedules itself independently.
- Each **runqueue** contains two priority arrays:
 - Two priority arrays (active, expired)
 - Tasks indexed by priority
 - When no more active(active array is empty), arrays are exchanged

Priorities and Time-slice length

<u>numeric priority</u>	<u>relative priority</u>		<u>time quantum</u>
0	highest		200 ms
• • • 99 100 • • • 140	lowest		10 ms

List of Tasks Indexed According to Priorities



Linux Scheduling (Cont.)

- Real-time scheduling according to POSIX.1b
 - Real-time tasks have static priorities
- All other tasks have dynamic priorities that are based on *nice* value plus or minus 5
 - Interactivity of task determines plus or minus
 - More interactive -> more minus
 - (A task's interactivity is determined by how long it has been sleeping while waiting for I/O.)
 - Priority recalculated when task expired
 - This exchanging arrays implements adjusted priorities

Exam Questions

1. Explain various steps involved in *change of a Process State*.
2. Define *Process States*.
3. What is a *process*? Write the difference between *process and program*.
4. Define *Process and Program*.
5. Compare & Contrast *Process & Thread*.
6. What is the difference between a "*thread*" and a "*process*"?
7. Compare the difference between a *thread and a process*. List the *system calls related to threads*.
8. Define a *thread*. What are the *uses of thread*?
9. What are the *reasons for Process Suspension*?
10. What is *process control block*? Explain its structure.

Exam Questions

11. Describe the *process state transition diagram*. Explain *Process Control Block*.
12. What are the *types of CPU Scheduling*?
13. What are *preemptive and non-preemptive scheduling policies*?
14. Compare *preemptive and non-preemptive CPU scheduling*.
15. What is *CPU scheduler*?
16. What is *throughput, turnaround time, waiting time and response time*?
17. Explain any three *CPU Scheduling algorithms*.

Exam Questions



18. Assume the following are the jobs to execute with one processor:

Job	Burst Time	Priority
1	10	3
2	1	1
3	2	3
4	1	4
5	5	2

The jobs are assumed to have arrived in the order 1, 2, 3, 4, 5.

- Give *Gantt-Chart* illustrating the execution of these jobs using *FCFS*, *RR (quantum =1)*, *Shortest Process Next*, *Shortest Remaining Time*.
- What is the *turn-around time*, *waiting time* of each job for each of the above scheduling algorithms?